

Regresión Lineal

Actividad con Scikit-learn

Versión Adaptada: Dr. Hugo García Tecocoatzi

En Machine Learning y en este notebook vamos a usar Scikit-learn

¿Para qué se usa scikit-learn?

Scikit-learn (Sklearn) es la biblioteca más útil y robusta para machine learning en Python. Proporciona una selección de herramientas eficientes para machine learning y modelado estadístico que incluyen clasificación, regresión, agrupamiento y reducción de dimensionalidad a través de una interfaz consistente en Python.

En esta ocasión lo usaremos para entrenar un modelo de regresión lineal

¿Para qué se usa la regresión lineal?

El análisis de regresión lineal se usa para predecir el valor de una variable basándose en el valor de otra variable. La variable que deseas predecir se llama variable dependiente. La variable que usas para predecir el valor de la otra variable se llama variable independiente (predictora).

Haciendo predicciones con una Regresión Lineal

Dado que la representación es una ecuación lineal, hacer predicciones es tan simple como resolver la ecuación para un conjunto específico de entradas.

Vamos a hacer esto concreto con un ejemplo. Imagina que estamos prediciendo el peso (y) a partir de la altura (x). Nuestra representación del modelo de regresión lineal para este problema sería:

$$y = B0 + B1 * x1$$

o

$$\text{peso} = B0 + B1 * \text{altura}$$

Donde B0 es el coeficiente de sesgo y B1 es una razón que relaciona el peso y la altura. Usamos una técnica de machine learning para encontrar un buen conjunto de valores de coeficientes. Una vez encontrados, podemos introducir diferentes valores de altura para predecir el peso.

Por ejemplo, usemos $B0 = 0.1$ y $B1 = 0.5$. Vamos a introducirlos y calcular el peso (en kilogramos) para una persona con una altura de 182 centímetros.

$$\text{peso} = 0.1 + 0.5 * 182$$

$$\text{peso} = 91.1$$

Puedes ver que la ecuación anterior podría representarse como una línea en dos dimensiones. B0 es nuestro punto de partida independientemente de la altura que tengamos. Podemos recorrer un montón de alturas de 100 a 250 centímetros e introducirlas en la ecuación y obtener valores de peso, creando nuestra línea.

Ahora que sabemos cómo hacer predicciones dado un modelo de regresión lineal aprendido, veamos algunas reglas generales para preparar nuestros datos y aprovechar al máximo este tipo de modelo.

Importar e Instalar Bibliotecas

```
::: { .cell _kg_hide-output='true' quarto-private-1='{ "key": "papermill", "value": { "duration": 10.251174, "end_time": "2022-02-21T21:06:34.673471", "exception": false, "start_time": "2022-02-21T21:06:24.422297", "status": "completed" } }' tags=[] execution_count=2 }
```

```
pip install hvplot
```

~C

Note: you may need to restart the kernel to use updated packages.

Collecting hvplot

Downloading hvplot-0.12.2-py3-none-any.whl.metadata (19 kB)

Collecting bokeh>=3.1 (from hvplot)

Downloading bokeh-3.10.0-py3-none-any.whl.metadata (11 kB)

Collecting colorcet>=2 (from hvplot)

Downloading colorcet-3.2.1-py3-none-any.whl.metadata (6.3 kB)

Collecting holoviews>=1.19.0 (from hvplot)

Downloading holoviews-1.23.2-py3-none-any.whl.metadata (7.8 kB)

Requirement already satisfied: numpy>=1.21 in c:\Users\tamar\AppData\Local\Programs\Python\Python310\lib\site-packages (from hvplot) (2.4.2)

Requirement already satisfied: packaging in C:\Users\tamar\AppData\Roaming\Python\Python313\packages (from hvplot) (26.0)

Requirement already satisfied: pandas>=1.3 in c:\Users\tamar\AppData\Local\Programs\Python\Python313\packages (from hvplot) (3.0.0)

Collecting panel>=1.0 (from hvplot)

 Downloading panel-1.9.4-py3-none-any.whl.metadata (15 kB)

Collecting param<3.0,>=1.12.0 (from hvplot)

 Downloading param-2.4.1-py3-none-any.whl.metadata (22 kB)

Collecting jinja2>=2.9 (from bokeh>=3.1->hvplot)

 Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)

Requirement already satisfied: narwhals>=1.13 in c:\Users\tamar\AppData\Local\Programs\Python\Python313\packages (from bokeh>=3.1->hvplot) (2.17.0)

Requirement already satisfied: pillow>=7.1.0 in c:\Users\tamar\AppData\Local\Programs\Python\Python313\packages (from bokeh>=3.1->hvplot) (12.1.1)

Collecting pyyaml>=3.10 (from bokeh>=3.1->hvplot)

 Using cached pyyaml-6.0.3-cp313-cp313-win_amd64.whl.metadata (2.4 kB)

Requirement already satisfied: tornado>=6.2 in C:\Users\tamar\AppData\Roaming\Python\Python313\packages (from bokeh>=3.1->hvplot) (6.5.4)

Collecting xyzservices>=2021.09.1 (from bokeh>=3.1->hvplot)

 Downloading xyzservices-2026.3.0-py3-none-any.whl.metadata (4.1 kB)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\tamar\AppData\Roaming\Python\Python313\packages (from holoviews>=1.19.0->hvplot) (2.9.0.post0)

Collecting pyviz-comms>=2.1 (from holoviews>=1.19.0->hvplot)

 Downloading pyviz_comms-3.0.6-py3-none-any.whl.metadata (7.7 kB)

Collecting MarkupSafe>=2.0 (from jinja2>=2.9->bokeh>=3.1->hvplot)

 Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)

Requirement already satisfied: tzdata in c:\Users\tamar\AppData\Local\Programs\Python\Python313\packages (from pandas>=1.3->hvplot) (2025.3)

Collecting bokeh>=3.1 (from hvplot)

 Downloading bokeh-3.9.2-py3-none-any.whl.metadata (10 kB)

Collecting linkify-it-py (from panel>=1.0->hvplot)

 Downloading linkify_it_py-2.2.0-py3-none-any.whl.metadata (8.6 kB)

Collecting markdown (from panel>=1.0->hvplot)

 Downloading markdown-3.10.3-py3-none-any.whl.metadata (5.1 kB)

Collecting markdown-it-py (from panel>=1.0->hvplot)

 Downloading markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)

Collecting mdit-py-plugins (from panel>=1.0->hvplot)

 Downloading mdit_py_plugins-0.6.1-py3-none-any.whl.metadata (2.9 kB)

Collecting nh3 (from panel>=1.0->hvplot)

 Downloading nh3-0.3.7-cp38-abi3-win_amd64.whl.metadata (2.0 kB)

Collecting panel-material-ui>=0.10.0 (from panel>=1.0->hvplot)

 Downloading panel_material_ui-0.14.2-py3-none-any.whl.metadata (7.4 kB)

Collecting requests (from panel>=1.0->hvplot)

```

Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: tqdm in c:\Users\tamar\AppData\Local\Programs\Python\Python313\
packages (from panel>=1.0->hvplot) (4.67.3)
Requirement already satisfied: typing-extensions in c:\Users\tamar\AppData\Local\Programs\Py
packages (from panel>=1.0->hvplot) (4.15.0)
Requirement already satisfied: contourpy>=1.2 in c:\Users\tamar\AppData\Local\Programs\Python
packages (from bokeh>=3.1->hvplot) (1.3.3)
Requirement already satisfied: six>=1.5 in C:\Users\tamar\AppData\Roaming\Python\Python313\s
packages (from python-dateutil>=2.8.2->holoviews>=1.19.0->hvplot) (1.17.0)
Collecting mdurl~=0.1 (from markdown-it-py->panel>=1.0->hvplot)
  Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Collecting charset_normalizer<4,>=2 (from requests->panel>=1.0->hvplot)
  Downloading charset_normalizer-3.5.1-cp313-cp313-win_amd64.whl.metadata (46 kB)
Collecting idna<4,>=2.5 (from requests->panel>=1.0->hvplot)
  Using cached idna-3.19-py3-none-any.whl.metadata (9.2 kB)
Collecting urllib3<3,>=1.26 (from requests->panel>=1.0->hvplot)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests->panel>=1.0->hvplot)
  Using cached certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: colorama in C:\Users\tamar\AppData\Roaming\Python\Python313\s
packages (from tqdm->panel>=1.0->hvplot) (0.4.6)
Downloading hvplot-0.12.2-py3-none-any.whl (180 kB)
Downloading param-2.4.1-py3-none-any.whl (151 kB)
Downloading colorcet-3.2.1-py3-none-any.whl (262 kB)
Downloading holoviews-1.23.2-py3-none-any.whl (6.0 MB)
----- 0.0/6.0 MB ? eta -:--:--
----- 3.1/6.0 MB 13.7 MB/s eta 0:00:01
----- 6.0/6.0 MB 19.3 MB/s 0:00:00
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading panel-1.9.4-py3-none-any.whl (30.3 MB)
----- 0.0/30.3 MB ? eta -:--:--
----- 5.8/30.3 MB 28.9 MB/s eta 0:00:01
----- 11.5/30.3 MB 28.8 MB/s eta 0:00:01
----- 18.6/30.3 MB 30.7 MB/s eta 0:00:01
----- 27.3/30.3 MB 33.5 MB/s eta 0:00:01
----- 30.3/30.3 MB 32.0 MB/s 0:00:00
Downloading bokeh-3.9.2-py3-none-any.whl (6.4 MB)
----- 0.0/6.4 MB ? eta -:--:--
----- 6.3/6.4 MB 29.8 MB/s eta 0:00:01
----- 6.4/6.4 MB 28.2 MB/s 0:00:00
Downloading panel_material_ui-0.14.2-py3-none-any.whl (1.7 MB)
----- 0.0/1.7 MB ? eta -:--:--

```

```

----- 1.7/1.7 MB 29.2 MB/s 0:00:00
Downloading pyviz_comms-3.0.6-py3-none-any.whl (84 kB)
Using cached pyyaml-6.0.3-cp313-cp313-win_amd64.whl (154 kB)
Downloading xyzservices-2026.3.0-py3-none-any.whl (94 kB)
Downloading linkify_it_py-2.2.0-py3-none-any.whl (21 kB)
Downloading markdown-3.10.3-py3-none-any.whl (110 kB)
Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Downloading mdit_py_plugins-0.6.1-py3-none-any.whl (66 kB)
Downloading nh3-0.3.7-cp38-abi3-win_amd64.whl (644 kB)
----- 0.0/644.5 kB ? eta -:--:--
----- 644.5/644.5 kB 13.8 MB/s 0:00:00
Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.1-cp313-cp313-win_amd64.whl (199 kB)
Using cached idna-3.19-py3-none-any.whl (68 kB)
Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: xyzservices, urllib3, pyyaml, param, nh3, mdurl, MarkupSafe,
it-py, idna, colorcet, charset_normalizer, certifi, requests, pyviz-comms, markdown-
it-py, jinja2, mdit-py-plugins, bokeh, panel-material-ui, panel, holoviews, hvplot

```

```

--- ----- 2/23 [pyyaml]
----- 3/23 [param]
----- 7/23 [markdown]
----- 10/23 [colorcet]
----- 13/23 [requests]
----- 15/23 [markdown-it-py]
----- 16/23 [jinja2]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 18/23 [bokeh]
----- 19/23 [panel-material-ui]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]

```

```

----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 20/23 [panel]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 21/23 [holoviews]
----- 22/23 [hvplot]
----- 23/23 [hvplot]

```

Successfully installed MarkupSafe-3.0.3 bokeh-3.9.2 certifi-2026.7.22 charset_normalizer-3.5.1 colorcet-3.2.1 holoviews-1.23.2 hvplot-0.12.2 idna-3.19 jinja2-3.1.6 linkify-it-py-2.2.0 markdown-3.10.3 markdown-it-py-4.2.0 mdit-py-plugins-0.6.1 mdurl-0.1.2 nh3-0.3.7 panel-1.9.4 panel-material-ui-0.14.2 param-2.4.1 pyviz-comms-3.0.6 pyyaml-6.0.3 requests-2.34.2 urllib3-2.7.0 xyzservices-2026.3.0

WARNING: The script markdown_py.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python113\Scripts'.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script idna.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python113\Scripts'.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script normalizer.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python113\Scripts'.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script markdown-it.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python113\Scripts'.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script bokeh.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python113\Scripts'.

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script panel.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python38-64\Scripts'

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

WARNING: The script holoviews.exe is installed in 'c:\Users\tamar\AppData\Local\Programs\Python\Python38-64\Scripts'

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

[notice] A new release of pip is available: 26.1.2 -> 26.2.1

[notice] To update, run: python.exe -m pip install --upgrade pip

:::

```
::: {".cell_cell_guid": "b1076dfc-b9ad-4769-8c92-a6c4dae69d19", ".uuid": "8f2839f25d086af736a60e9eeb907d3b93b6e0",
quarto-private-1": {"key": "papermill", "value": {"duration": 4.231627, "end_time": "2022-02-21T21:06:38.945432",
"exception": false, "start_time": "2022-02-21T21:06:34.713805", "status": "completed"}}}
tags": [], "execution_count": 3}
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import hvplot.pandas

from sklearn.model_selection import train_test_split

from sklearn import metrics

from sklearn.linear_model import LinearRegression

%matplotlib inline
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): application/javascript, application/vnd.holoviews

Unable to display output for mime type(s): application/javascript, application/vnd.holoviews

Unable to display output for mime type(s): application/vnd.holoviews_exec.v0+json, text/html

:::

Explorar los Datos

```
df=pd.read_csv('Realestate.csv')
```

```
df.head()
```

	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of conven
0	1	2012.917	32.0	84.87882	10
1	2	2012.917	19.5	306.59470	9
2	3	2013.583	13.3	561.98450	5
3	4	2013.500	13.3	561.98450	5
4	5	2012.833	5.0	390.56840	5

Tenemos los precios en dólar por unidad de área de casas en Real Estate.

```
df.shape
```

```
(414, 8)
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 414 entries, 0 to 413
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	No	414 non-null	int64
1	X1 transaction date	414 non-null	float64
2	X2 house age	414 non-null	float64
3	X3 distance to the nearest MRT station	414 non-null	float64
4	X4 number of convenience stores	414 non-null	int64
5	X5 latitude	414 non-null	float64
6	X6 longitude	414 non-null	float64
7	Y house price of unit area	414 non-null	float64

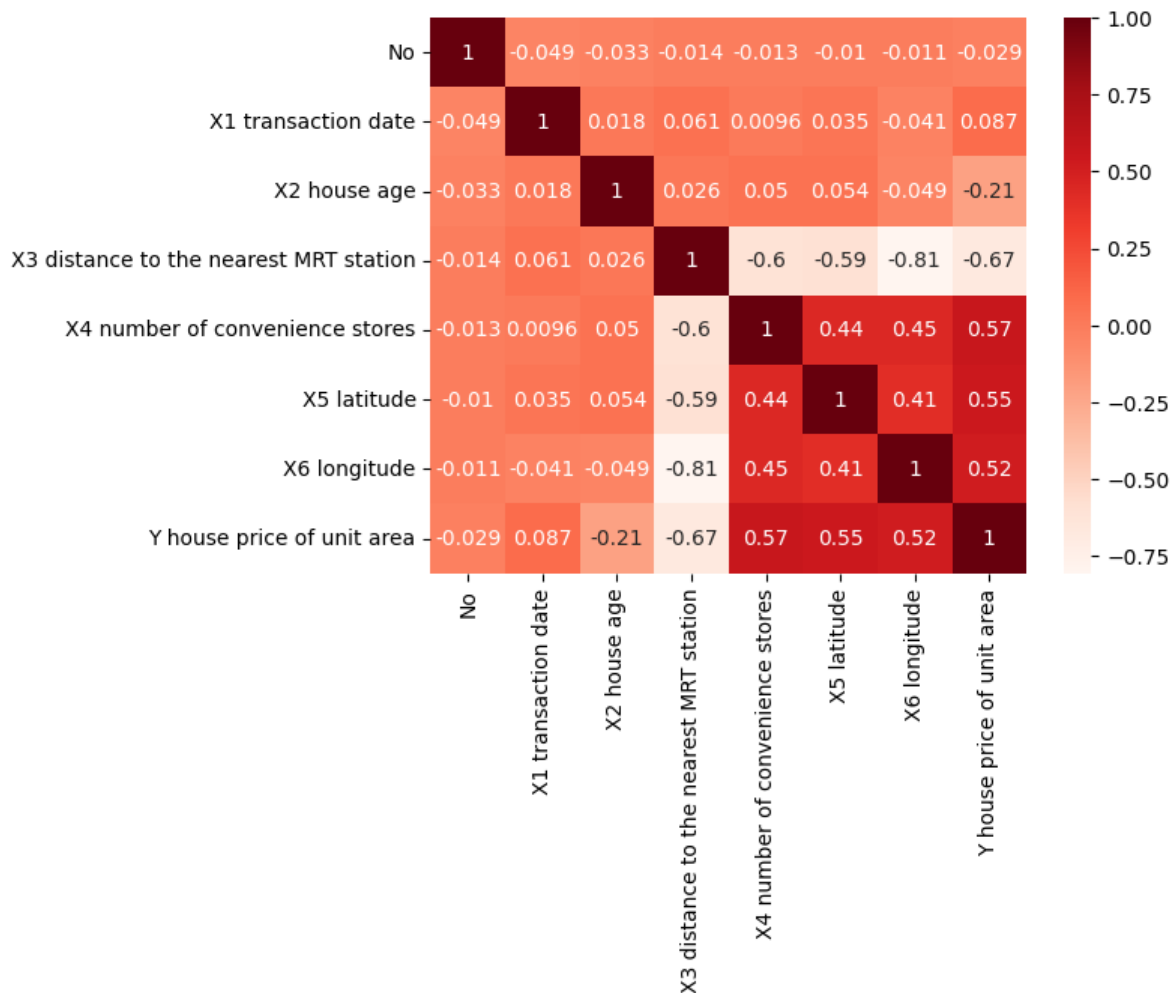
```
dtypes: float64(6), int64(2)
```

```
memory usage: 26.0 KB
```

```
df.corr()
```

	No	X1 transaction date	X2 house age	X3 distance to the
No	1.000000	-0.048658	-0.032808	-0.013573
X1 transaction date	-0.048658	1.000000	0.017549	0.060880
X2 house age	-0.032808	0.017549	1.000000	0.025622
X3 distance to the nearest MRT station	-0.013573	0.060880	0.025622	1.000000
X4 number of convenience stores	-0.012699	0.009635	0.049593	-0.602519
X5 latitude	-0.010110	0.035058	0.054420	-0.591067
X6 longitude	-0.011059	-0.041082	-0.048520	-0.806317
Y house price of unit area	-0.028587	0.087491	-0.210567	-0.673613

```
sns.heatmap(df.corr(), annot=True,cmap='Reds')  
plt.show()
```

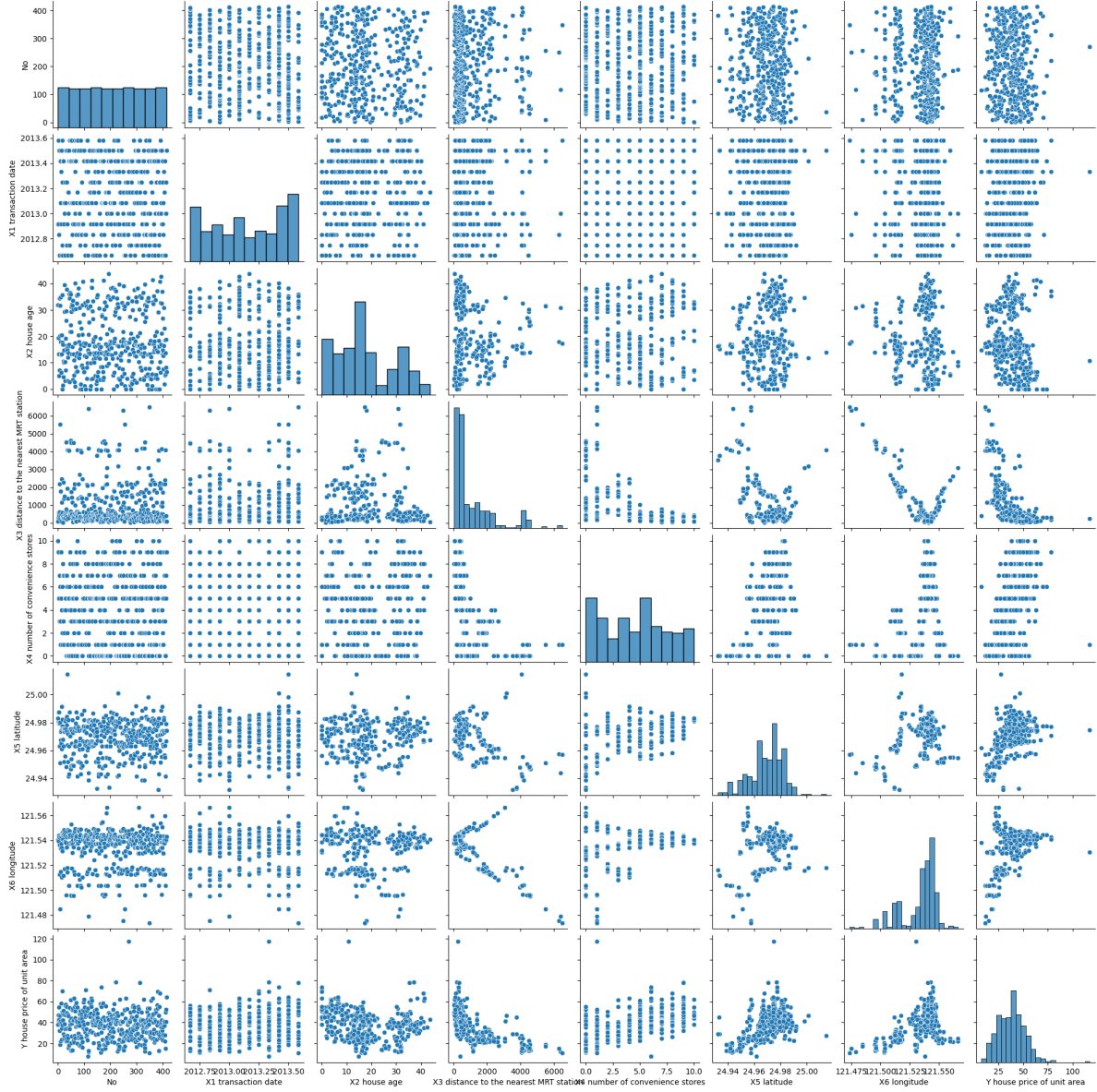


1. Describe qué información lees de esto. Gracias a estas últimas líneas de código se pueden analizar las correlaciones entre las variables de la base de datos, específicamente, encuentro más útil el último mapa que representa matriz de correlaciones entre las variables, ya que es mucho más atractiva visualmente y más fácil de interpretar por ende.
2. ¿Cuáles son las variables fuertemente correlacionadas? las más fuertes son:
 - a. la longitud y la distancia a la estación de MRT más cercana tienen una correlación negativa fuerte (-0.81)
 - b. el precio por unidad y la distancia a la estación de MRT más cercana también tienen una correlación negativa fuerte (-0.67)
 - c. la latitud y la distancia a la estación de MRT más cercana (-0.59)
 - d. el número de tiendas de conveniencia y la distancia a la estación de MRT más cercana (-0.6)

- e. número de tiendas de conveniencia y el precio por unidad de área (0.57)
 - f. latitud/longitud y precio por unidad de área (0.55 y 0.52 respectivamente)
3. ¿Te hacen sentido? las que si me hacen sentido: f, e, d y b las que no me hacen tanto sentido: a y c

Análisis Exploratorio de Datos (EDA)

```
sns.pairplot(df)
plt.show()
```



4. Con base en la correlación y estas gráficas, ¿qué variables pueden modelarse con una regresión lineal? Para la variable objetivo (precio de unidad de área) house price of unit como variable objetivo y distancia al MRT más cercano como variable predictora (negativa muy leve) house price of unit como variable objetivo y latitud como variable predictora house price of unit como variable objetivo y longitud como variable predictora

Entrenando un Modelo de Regresión Lineal

Arrays X y y

Vamos a quitar el precio por unidad de área. La variable a predecir es el número de tiendas de convivencia cercanas

```
X=df.drop('Y house price of unit area', axis=1)
```

```
y=df['Y house price of unit area']
```

```
print("X=",X.shape,"\ny=", y.shape)
```

```
X= (414, 7)
```

```
y= (414,)
```

División de los datos: Entrenamiento y Prueba

Ahora dividamos los datos en un conjunto de entrenamiento y un conjunto de prueba. Entrenaremos nuestro modelo en el conjunto de entrenamiento y luego usaremos el conjunto de prueba para evaluar el modelo.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=101)
```

```
X_train.shape
```

```
(289, 7)
```

```
X_test.shape
```

```
(125, 7)
```

Ahora vamos a usar la función predeterminada de Scikit-learn: LinearRegression()

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

fit_intercept	fit_intercept:	True
	bool, default=True	

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

copy_X	copy_X: bool,	True
	default=True	

If True, X will be copied; else, it may be overwritten.

tol	tol: float, default=1e-6	1e-06
-----	--------------------------	-------

The precision of the solution ('coef_') is determined by 'tol' which specifies a different convergence criterion for the 'lsqr' solver.

'tol' is set as 'atol' and 'btol' of :func:'scipy.sparse.linalg.lsqr' when fitting on sparse training data. This parameter has no effect when fitting on dense data.

.. versionadded:: 1.7

<code>n_jobs</code> <code>n_jobs: int,</code> <code>default=None</code>	None
The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly <code>'n_targets > 1'</code> and secondly <code>'X'</code> is sparse or if <code>'positive'</code> is set to <code>'True'</code>. “None“ means 1 unless in a <code>:obj:'joblib.parallel_backend'</code> context. “-1“ means using all processors. See <code>:term:'Glossary '</code> for more details.	
<code>positive</code> <code>positive: bool,</code> <code>default=False</code>	False
When set to “True“, forces the coefficients to be positive. This option is only supported for dense arrays.	
For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see <code>:ref:'sphx_glr_auto_examples_linear_model_plot_nnls.py'</code>.	
.. versionadded:: 0.24	

Evaluación del modelo

```
model.coef_
```

```
array([-2.92429686e-03,  4.68668437e+00, -2.75675355e-01, -4.24850544e-03,  
       1.17832502e+00,  2.40947692e+02,  1.86694591e+01])
```

```
pd.DataFrame(model.coef_, X.columns, columns=['Coeficientes'])
```

	Coeficientes
No	-0.002924
X1 transaction date	4.686684
X2 house age	-0.275675
X3 distance to the nearest MRT station	-0.004249
X4 number of convenience stores	1.178325
X5 latitude	240.947692
X6 longitude	18.669459

5. Regresa a la pregunta 4, y ve si los coeficientes corresponden la figura. Los signos de los coeficientes coinciden con lo observado en la matriz de dispersión de la pregunta 4. La antigüedad y distancia al metro presentan coeficientes de signo negativo congruentes con la caída de los precios en sus gráficas. Por otro lado las tiendas de conveniencia, la longitud y la latitud (posición geográfica) muestran relaciones positivas acordes a la concentración de mayor valor en ciertas zonas urbanas, sin embargo el número identificador es tan cercano a cero debido a su falta de correlación visual y lógica.
6. ¿Cómo se interpreta cada coeficiente? Cada coeficiente corresponde, matemáticamente, a cada B_i lo que representa la razón de cambio de la variable objetivo por cada unidad de la variable predictora. Si es negativo, la relación entre las variables es inversamente proporcional, mientras que si es positivo es directamente proporcional.

Predicciones del modelo

```
y_pred = model.predict(X_test)
```

Métricas de Evaluación de Regresión

Aquí están las tres métricas de evaluación más comunes para problemas de regresión:

- **Error Absoluto Medio (MAE)** es la media del valor absoluto de los errores:

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Error Cuadrático Medio (MSE)** es la media de los errores al cuadrado:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Raíz del Error Cuadrático Medio (RMSE)** es la raíz cuadrada de la media de los errores al cuadrado:

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Comparando estas métricas: - **MAE** es la más fácil de entender, porque es el error promedio. - **MSE** es más popular que MAE, porque MSE “castiga” los errores más grandes, lo que tiende a ser útil en el mundo real. - **RMSE** es aún más popular que MSE, porque RMSE es interpretable en las unidades de “y”.

Todas estas son **funciones de pérdida**, porque queremos minimizarlas.

```
MAE= metrics.mean_absolute_error(y_test, y_pred)
MSE=metrics.mean_squared_error(y_test, y_pred)
RMSE= np.sqrt(MSE)
```

MAE

5.373024532570965

MSE

45.880307428741155

RMSE

np.float64(6.773500382279546)

Histograma de Residuos

- A menudo, para la Regresión Lineal, es una buena idea evaluar los residuos

$$(y - \hat{y})$$

por separado y no solo calcular métricas de rendimiento (ej. RMSE).

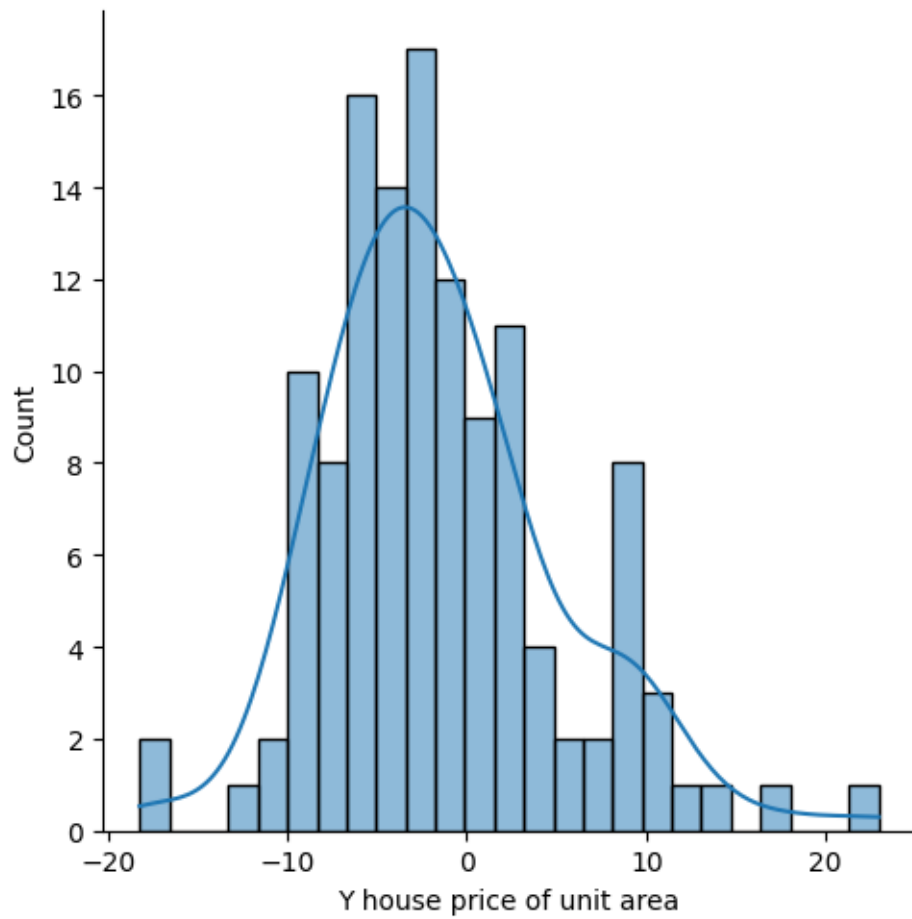
- Exploremos por qué esto es importante...
- Los errores residuales deberían ser aleatorios y aproximarse a una distribución normal.

```
test_residual= y_test - y_pred
```

```
pd.DataFrame({'Error Values': (test_residual)}).hvplot.kde()
```

```
:Distribution    [Error Values]    (Density)
```

```
sns.displot(test_residual, bins=25, kde=True)  
plt.show()
```



- El gráfico de residuos muestra el error residual vs. el valor verdadero de y.

```
sns.scatterplot(x=y_test, y=test_residual)

plt.axhline(y=0, color='r', ls='--')
plt.show()
```

